Django Complete Notes — A to Z

Beginner to Advanced | Python Backend Development

Language: Bangla + English

Framework: Django

API Framework: Django REST Framework (DRF)

Database: SQLite / PostgreSQL

Goal: Backend development, real-world projects, and interview preparation

---

Table of Contents

1. Introduction to Django
2. Python Prerequisites
3. Environment Setup
4. Project and App Structure
5. URL Routing
6. Views — Beginner to Advanced
7. Templates and Template Language
8. Static and Media Files
9. Models and Database
10. Django ORM — Beginner to Advanced
11. CRUD Operations
12. Forms and ModelForms
13. Django Admin
14. Authentication and Authorization
15. Sessions and Cookies
16. Middleware
17. Model Relationships
18. Query Optimization
19. Django REST Framework
20. Serializers and Validation
21. API Views, ViewSets and Routers
22. API Authentication and Permissions
23. Filtering, Searching, Ordering and Pagination

24. Signals, Custom Managers and Commands
25. Error Handling and Logging
26. Testing and Debugging
27. Django Security
28. Caching
29. Background Tasks with Celery
30. Email Integration
31. File Uploads and Storage
32. Environment Variables and Configuration
33. Git and GitHub Workflow
34. Production Deployment
35. Performance Optimization
36. Docker and CI/CD
37. Advanced Django Concepts
38. Practical Projects
39. Interview Questions
40. Final Checklist

Chapter 1: Introduction to Django

1.1 What is Django?

Django হলো Python-এর একটি high-level web framework। এটি দিয়ে secure, maintainable এবং scalable web application তৈরি করা যায়।

Django-এর প্রধান features:

- URL routing
- ORM এবং database migrations
- Template engine
- Forms এবং validation
- Authentication ও permissions
- Admin dashboard
- Middleware
- Security protections
- Testing tools
- Internationalization

- Caching support

1.2 Django Architecture — MVT

Django সাধারণত MVT architecture ব্যবহার করে।

- **Model:** Database structure এবং data-related behavior।
- **View:** Request processing এবং application logic।
- **Template:** HTML presentation।
- **URLconf:** URL-এর সঙ্গে view connect করে।

Request flow:

```
Browser
   ↓
URL Dispatcher
   ↓
Middleware
   ↓
View
   ↓
Model / Database (if needed)
   ↓
Template or JSON Response
   ↓
Middleware
   ↓
Browser
```

1.3 Django vs Flask vs FastAPI

Feature	Django	Flask	FastAPI
Type	Full-featured framework	Lightweight framework	API-focused framework
ORM	Built-in	External integration	External integration
Admin	Built-in	Extensions	Extensions

Forms	Built-in forms	Extensions	API validation
API	DRF available	Extensions	Strong API support
Best use	Full web applications	Flexible small apps	Modern APIs

1.4 When should you use Django?

Django উপযোগী:

- Blog এবং CMS
- E-commerce backend
- Inventory management
- School management
- Job portal
- Authentication-based applications
- REST APIs
- Admin-heavy business applications

Chapter 2: Python Prerequisites

Django শেখার আগে Python-এর নিচের বিষয়গুলো শিখতে হবে।

2.1 Core Python

- Variables and data types
- Operators
- Conditions
- Loops
- Functions
- Lists, tuples, sets, dictionaries
- List and dictionary comprehensions
- Modules and packages

- File handling
- Exception handling
- Classes and objects
- Inheritance and encapsulation
- Iterators and generators
- Decorators
- Context managers
- Virtual environments
- Basic SQL and HTTP

2.2 Functions

```
def calculate_total(price, quantity):
    return price * quantity


total = calculate_total(100, 3)
print(total)
```

2.3 Classes and Objects

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def get_price(self):
        return self.price


product = Product("Keyboard", 1500)
print(product.get_price())
```

2.4 Exception Handling

```
try:
    number = int("100")
except ValueError:
    print("Invalid number")
```

2.5 Virtual Environments

Virtual environment project-specific Python dependencies আলাদা রাখে।

```
python -m venv .venv
```

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Linux/macOS:

```
source .venv/bin/activate
```

Chapter 3: Environment Setup

3.1 Install Django

```
python -m pip install django
python -m django --version
```

3.2 Create a Project

```
django-admin startproject config .
```

Run the development server:

```
python manage.py runserver
```

Browser:

```
http://127.0.0.1:8000/
```

Development server production deployment-এর জন্য নয়।

3.3 Create an App

```
python manage.py startapp core
```

`settings.py`:

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
```

```
    "core",
]
```

3.4 Useful Commands

```
python manage.py check
python manage.py runserver
python manage.py startapp accounts
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py shell
python manage.py test
python manage.py showmigrations
python manage.py check --deploy
```

Chapter 4: Project and App Structure

4.1 Project Structure

```
project/
├── manage.py
├── config/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── core/
│   ├── migrations/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
├── templates/
├── static/
├── media/
├── requirements.txt
└── .gitignore
```

4.2 Important Files

- `manage.py` : Django management commands।
- `settings.py` : Project configuration।
- `urls.py` : URL routing।
- `models.py` : Database models।
- `views.py` : Request processing।
- `forms.py` : Forms and validation।
- `admin.py` : Admin customization।
- `tests.py` : Automated tests।
- `migrations/` : Database schema changes।
- `wsgi.py` : WSGI deployment interface।
- `asgi.py` : ASGI deployment interface।

4.3 Project vs App

Project: পুরো website বা application configuration।

App: একটি নির্দিষ্ট functionality-এর module, যেমন `accounts`, `products`, `orders`।

একটি project-এ একাধিক app থাকতে পারে।

Chapter 5: URL Routing

5.1 Basic URL Configuration

`core/urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
    path("about/", views.about, name="about"),
]
```

Project `config/urls.py`:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
```

```
    path("", include("core.urls")),
]
```

5.2 Dynamic URLs

```
path(
    "products/<int:product_id>/",
    views.product_detail,
    name="product_detail",
)
```

View:

```
from django.http import HttpResponse

def product_detail(request, product_id):
    return HttpResponse(f"Product ID: {product_id}")
```

Common path converters:

- `str`
- `int`
- `slug`
- `uuid`
- `path`

5.3 URL Reversing

Hard-coded URL-এর পরিবর্তে URL name ব্যবহার করো।

Template:

```
<a href="{% url 'home' %}">Home</a>
```

Python:

```
from django.urls import reverse

url = reverse("home")
```

`reverse_lazy()` class-based views-এর মতো deferred URL resolution-এর ক্ষেত্রে কাজে লাগে।

5.4 App URL Namespaces

`core/urls.py`:

```
from django.urls import path
from . import views
```

```
app_name = "core"

urlpatterns = [
    path("", views.home, name="home"),
]
```

এখন URL reverse করা যাবে:

```
reverse("core:home")
```

একাধিক app-এ একই নামের URL থাকলে namespace conflict কমাতে সাহায্য করে।

Chapter 6: Django Views — Beginner to Advanced

6.1 What Is a View?

View হলো Python function বা class, যা HTTP request process করে HTTP response দেয়।

View-এর কাজ:

- Request গ্রহণ করা
- Input validation করা
- Business logic চালানো
- Database থেকে data নেওয়া বা পরিবর্তন করা
- Template render করা
- JSON response দেওয়া
- Redirect বা error response দেওয়া

6.2 Function-Based Views (FBV)

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Django!")
```

URL:

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
]
```

`request` হলো incoming HTTP request object।

6.3 Render a Template

```
from django.shortcuts import render

def home(request):
    context = {
        "title": "Django Home",
        "message": "Learn Django step by step",
    }
    return render(request, "home.html", context)
```

`templates/home.html`:

```
<!DOCTYPE html>
<html>
<head>
    <title>{{ title }}</title>
</head>
<body>
    <h1>{{ message }}</h1>
</body>
</html>
```

6.4 HttpResponse vs render vs redirect

Function	Purpose
`HttpResponse()`	সরাসরি response পাঠায়
`render()`	Template render করে response দেয়
`redirect()`	Browser-কে অন্য URL-এ পাঠায়

`JsonResponse()`	JSON response দেয়
`get_object_or_404()`	Object না পেলে 404 raise করে

Redirect example:

```
from django.shortcuts import redirect

def go_home(request):
    return redirect("home")
```

6.5 Request Object

Property	Purpose
`request.method`	HTTP method
`request.GET`	Query-string data
`request.POST`	Form data
`request.FILES`	Uploaded files
`request.user`	Current user
`request.path`	Requested path
`request.headers`	HTTP headers

`request.COOKIES`	Cookies

GET example:

```
def search(request):
    query = request.GET.get("q", "")
    return HttpResponse(f"Search: {query}")
```

URL:

```
/search/?q=django
```

POST example:

```
def contact(request):
    if request.method == "POST":
        name = request.POST.get("name", "")
        return HttpResponse(f"Hello {name}")

    return HttpResponse("Submit the form")
```

Production code-এ Django Forms বা serializers দিয়ে submitted data validate করো।

6.6 HTTP Methods

Method	**Typical purpose**
GET	Data retrieve
POST	Submit/create
PUT	Full update
PATCH	Partial update

DELETE	Delete resource

GET request দিয়ে database state পরিবর্তন করা উচিত নয়।

6.7 Database List View

```
from django.shortcuts import render
from .models import Product

def product_list(request):
    products = Product.objects.all()
    return render(
        request,
        "products/list.html",
        {"products": products},
    )
```

Template:

```
{% for product in products %}
    <h2>{{ product.name }}</h2>
    <p>{{ product.price }}</p>
{% empty %}
    <p>No products found.</p>
{% endfor %}
```

6.8 Detail View

```
from django.shortcuts import get_object_or_404, render
from .models import Product

def product_detail(request, pk):
    product = get_object_or_404(Product, pk=pk)
    return render(
        request,
        "products/detail.html",
        {"product": product},
    )
```

6.9 Create and Update Views

`forms.py`:

```
from django import forms
from .models import Product

class ProductForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = ["name", "price", "description"]
```

Create view:

```
from django.shortcuts import redirect, render
from .forms import ProductForm

def product_create(request):
    if request.method == "POST":
        form = ProductForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("product_list")
    else:
        form = ProductForm()

    return render(
        request,
        "products/form.html",
        {"form": form},
    )
```

Update view:

```
from django.shortcuts import get_object_or_404, redirect, render
from .forms import ProductForm
from .models import Product

def product_update(request, pk):
    product = get_object_or_404(Product, pk=pk)

    if request.method == "POST":
        form = ProductForm(request.POST, instance=product)

        if form.is_valid():
            form.save()
```

```
            return redirect("product_list")
    else:
        form = ProductForm(instance=product)

    return render(
        request,
        "products/form.html",
        {"form": form},
    )
```

6.10 Delete View

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import get_object_or_404, redirect
from django.views.decorators.http import require_POST
from .models import Product

@login_required
@require_POST
def product_delete(request, pk):
    product = get_object_or_404(Product, pk=pk)

    if not request.user.has_perm("core.delete_product"):
        from django.core.exceptions import PermissionDenied
        raise PermissionDenied

    product.delete()
    return redirect("product_list")
```

Delete form:

```
<form method="post" action="{% url 'product_delete' product.pk %}">
    {% csrf_token %}
    <button type="submit">Delete</button>
</form>
```

Production-এ object ownership এবং business-specific authorization-ও যাচাই করতে হবে।

6.11 Class-Based Views (CBV)

```
from django.http import HttpResponse
from django.views import View

class HomeView(View):
```

```
    def get(self, request):
        return HttpResponse("Welcome!")

    def post(self, request):
        return HttpResponse("POST received")
```

URL:

```
path("", HomeView.as_view(), name="home")
```

`as_view()` class-টিকে URL configuration-এ ব্যবহারযোগ্য callable view-তে পরিণত করে।

6.12 Generic Class-Based Views

ListView

```
from django.views.generic import ListView
from .models import Product

class ProductListView(ListView):
    model = Product
    template_name = "products/list.html"
    context_object_name = "products"
    paginate_by = 10
```

DetailView

```
from django.views.generic import DetailView

class ProductDetailView(DetailView):
    model = Product
    template_name = "products/detail.html"
    context_object_name = "product"
```

CreateView

```
from django.urls import reverse_lazy
from django.views.generic import CreateView

class ProductCreateView(CreateView):
    model = Product
    fields = ["name", "price", "description"]
    template_name = "products/form.html"
    success_url = reverse_lazy("product_list")
```

UpdateView

```
from django.views.generic import UpdateView

class ProductUpdateView(UpdateView):
    model = Product
    fields = ["name", "price", "description"]
    template_name = "products/form.html"
    success_url = reverse_lazy("product_list")
```

DeleteView

```
from django.views.generic import DeleteView

class ProductDeleteView(DeleteView):
    model = Product
    template_name = "products/confirm_delete.html"
    success_url = reverse_lazy("product_list")
```

DeleteView-এর confirmation template-এ POST form এবং CSRF token থাকা উচিত। Permission check করতে `dispatch()` বা উপযুক্ত mixin override করা যায়।

6.13 Important CBV Methods

- `dispatch()` : HTTP method অনুযায়ী handler select করে।
- `get_queryset()` : QuerySet নির্ধারণ করে।
- `get_context_data()` : Template context পরিবর্তন করে।
- `get_object()` : Detail/update/delete-এর object নির্ধারণ করে।
- `form_valid()` : Valid form-এর behavior নির্ধারণ করে।
- `form_invalid()` : Invalid form-এর behavior নির্ধারণ করে।

6.14 View Decorators

Login required:

```
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

POST-only:

```
from django.views.decorators.http import require_POST

@require_POST
def submit_order(request):
```

```
    # Validate and process order.
    ...
```

Permission required:

```
from django.contrib.auth.decorators import permission_required

@permission_required("core.view_product", raise_exception=True)
def product_report(request):
    ...
```

6.15 JSON Response

```
from django.http import JsonResponse

def health_check(request):
    return JsonResponse({
        "status": "ok",
        "message": "Application is running",
    })
```

Complex APIs-এর জন্য DRF সাধারণত বেশি উপযোগী।

6.16 Error Responses

Code	Meaning
200	OK
201	Created
302	Redirect
400	Bad Request
401	Unauthenticated
403	Forbidden
404	Not Found

405	Method Not Allowed
429	Too Many Requests
500	Internal Server Error

6.17 FBV vs CBV

Feature	FBV	CBV
Structure	Function	Class
Simplicity	সহজ	Generic patterns-এ কার্যকর
Reuse	Functions/ decorators	Inheritance/mixins
Custom logic	সরাসরি	Methods override
Good for	Custom workflows	Repeated CRUD patterns

6.18 View Testing

```
from django.test import TestCase
from django.urls import reverse
```

```
class HomeViewTests(TestCase):
    def test_home_page(self):
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)
```

6.19 View Best Practices

1. View ছোট ও পরিষ্কার রাখো।
2. Forms বা serializers দিয়ে input validate করো।
3. Missing object handle করতে `get_object_or_404()` ব্যবহার করো।
4. Named URLs ব্যবহার করো।
5. State-changing action GET দিয়ে করো না।
6. CSRF protection বজায় রাখো।
7. Authentication এবং authorization দুটোই যাচাই করো।
8. N+1 queries এড়িয়ে চলো।
9. Business logic reusable function/service-এ রাখো।
10. Automated tests লেখো।

Chapter 7: Templates and Template Language

7.1 Template Configuration

`settings.py`:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]
```

7.2 Variables

```
<h1>{{ title }}</h1>
<p>{{ user.username }}</p>
```

7.3 Conditions

```
{% if user.is_authenticated %}
    <p>Welcome, {{ user.username }}</p>
{% else %}
    <p>Please log in.</p>
{% endif %}
```

7.4 Loops

```
{% for product in products %}
    <p>{{ product.name }}</p>
{% empty %}
    <p>No products found.</p>
{% endfor %}
```

7.5 Filters

```
{{ name|title }}
{{ description|truncatewords:20 }}
{{ created_at|date:"Y-m-d" }}
{{ price|floatformat:2 }}
```

7.6 Template Inheritance

`base.html`:

```
<!DOCTYPE html>
<html>
<head>
    <title>{% block title %}My Site{% endblock %}</title>
</head>
<body>
    {% block content %}{% endblock %}
</body>
</html>
```

`home.html`:

```
{% extends "base.html" %}
```

```
{% block title %}Home{% endblock %}

{% block content %}
    <h1>Welcome</h1>
{% endblock %}
```

7.7 Include Reusable Templates

```
{% include "partials/navbar.html" %}
```

7.8 Context Processors

Context processor template-এ globally available data যোগ করতে পারে। অতিরিক্ত database query চালায় এমন context processor অপ্রয়োজনীয়ভাবে ব্যবহার করো না।

7.9 Custom Template Tags and Filters

Custom filter বা tag reusable presentation logic-এর জন্য ব্যবহার করা যায়। Business logic template-এর মধ্যে জটিলভাবে লিখবে না।

7.10 Template Security

Django template defaultভাবে variable output escape করে।

Untrusted user input-এর ক্ষেত্রে অপ্রয়োজনীয়ভাবে `safe` filter ব্যবহার করবে না।

Chapter 8: Static and Media Files

8.1 Static Files

CSS, JavaScript এবং fixed site images static files।

`settings.py`:

```
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
```

Template:

```
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">
<script src="{% static 'js/app.js' %}" defer></script>
```

8.2 Media Files

User-uploaded files-এর জন্য:

```
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Development URL configuration:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # Other URL patterns
]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT,
    )
```

এই helper production media serving-এর জন্য নয়।

8.3 Static vs Media

Static	Media
Developer-provided files	User-uploaded files
CSS, JS, logos	Profile photos, documents
`STATIC_ROOT`	`MEDIA_ROOT`
`collectstatic`	Storage backend

Production-এ static ও media storage আলাদাভাবে configure করো।

Chapter 9: Models and Database

9.1 What Is a Model?

Django model database data-এর structure এবং related behavior define করে।

`models.py`:

```
from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=200)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    description = models.TextField(blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

9.2 Common Field Types

Field	Purpose
`CharField`	Short text
`TextField`	Long text
`IntegerField`	Integer
`DecimalField`	Exact decimal values
`FloatField`	Floating-point values
`BooleanField`	True/False

`DateField`	Date
`DateTimeField`	Date and time
`EmailField`	Email format validation
`URLField`	URL validation
`FileField`	File storage reference
`ImageField`	Image upload field
`JSONField`	JSON-compatible data
`UUIDField`	UUID value

9.3 Field Options

- `null=True` : Database NULL অনুমোদন করে।
- `blank=True` : Validation-এ empty value অনুমোদন করে।
- `default` : Default value।
- `unique=True` : Unique constraint।
- `db_index=True` : Index তৈরি করতে পারে।
- `choices` : Allowed options নির্ধারণ করে।
- `editable=False` : সাধারণ model form/admin editing থেকে field বাদ দিতে সাহায্য করে।
- `help_text` : Admin/form-এ help text।

`null` database-level এবং `blank` validation-level concept।

9.4 Choices

```
class Order(models.Model):
    class Status(models.TextChoices):
        PENDING = "pending", "Pending"
        PAID = "paid", "Paid"
        CANCELLED = "cancelled", "Cancelled"

    status = models.CharField(
        max_length=20,
        choices=Status.choices,
        default=Status.PENDING,
    )
```

9.5 Model Meta

```
class Product(models.Model):
    name = models.CharField(max_length=200)

    class Meta:
        ordering = ["name"]
        verbose_name = "Product"
        verbose_name_plural = "Products"
```

`Meta` class ordering, constraints, indexes এবং model metadata configure করতে পারে।

9.6 Model Validation

`clean()` method দিয়ে cross-field validation লেখা যায়। Model validation স্বয়ংক্রিয়ভাবে প্রতিটি `save()` call-এ চলে না; প্রয়োজন অনুযায়ী `full_clean()` বা form/serializer validation ব্যবহার করতে হয়।

9.7 Migrations

```
python manage.py makemigrations
python manage.py migrate
python manage.py showmigrations
python manage.py sqlmigrate core 0001
```

- `makemigrations` : Model changes থেকে migration তৈরি করে।
- `migrate` : Database-এ migration প্রয়োগ করে।
- `showmigrations` : Migration status দেখায়।

- `sqlmigrate` : Migration-এর SQL দেখায়।

Migration files সাধারণত Git-এ commit করতে হয়।

Chapter 10: Django ORM — Beginner to Advanced

ORM Python code দিয়ে database query করার সুবিধা দেয়।

10.1 Basic Queries

```
from core.models import Product

Product.objects.all()
Product.objects.filter(is_active=True)
Product.objects.exclude(is_active=False)
Product.objects.get(pk=1)
Product.objects.first()
Product.objects.last()
Product.objects.count()
Product.objects.exists()
Product.objects.order_by("-created_at")
```

10.2 `get()` vs `filter()`

- `get()` : একটি object প্রত্যাশা করে।
- Object না পেলে `DoesNotExist` হতে পারে।
- একাধিক object match করলে `MultipleObjectsReturned` হতে পারে।
- `filter()` : QuerySet return করে।
- কোনো match না থাকলে empty QuerySet return করে।

10.3 Field Lookups

```
Product.objects.filter(price__gt=100)
Product.objects.filter(price__gte=100)
Product.objects.filter(price__lt=500)
Product.objects.filter(name__icontains="phone")
Product.objects.filter(name__startswith="Sam")
Product.objects.filter(created_at__year=2026)
```

Common lookups:

- `exact`
- `iexact`
- `contains`
- `icontains`

- `gt`, `gte`
- `lt`, `lte`
- `in`
- `range`
- `isnull`
- `startswith`, `istartswith`

10.4 Create

```
Product.objects.create(
    name="Keyboard",
    price="1500.00",
)
```

অথবা:

```
product = Product(
    name="Mouse",
    price="800.00",
)
product.save()
```

10.5 Update and Delete

```
Product.objects.filter(pk=1).update(price="1800.00")
```

```
product = Product.objects.get(pk=1)
product.price = "1800.00"
product.save()
```

```
Product.objects.filter(pk=1).delete()
```

`QuerySet.update()` সাধারণ model `save()` method call করে না এবং সংশ্লিষ্ট save signals-ও সাধারণত trigger করে না।

10.6 Q Objects

OR, AND এবং NOT logic-এর জন্য:

```
from django.db.models import Q

products = Product.objects.filter(
    Q(name__icontains="phone") |
    Q(description__icontains="phone")
)
```

10.7 F Expressions

Database-এর বর্তমান field value ব্যবহার করে update:

```
from django.db.models import F

Product.objects.filter(pk=1).update(
    price=F("price") + 100
)
```

10.8 Aggregation

```
from django.db.models import Avg, Count, Max, Min, Sum

Product.objects.aggregate(
    average_price=Avg("price"),
    total_products=Count("id"),
    highest_price=Max("price"),
    lowest_price=Min("price"),
    total_price=Sum("price"),
)
```

10.9 `values()` and `values_list()`

```
Product.objects.values("id", "name", "price")
```

```
Product.objects.values_list("name", flat=True)
```

শুধু প্রয়োজনীয় fields retrieve করতে এগুলো উপকারী।

10.10 `annotate()`

প্রতিটি row-এর সঙ্গে calculated value যুক্ত করতে:

```
from django.db.models import Count

categories = Category.objects.annotate(
    product_count=Count("products")
)
```

10.11 `distinct()`

Duplicate results বাদ দিতে:

```
Product.objects.values("name").distinct()
```

Database এবং query structure অনুযায়ী distinct behavior ভিন্ন হতে পারে।

10.12 `get_or_create()` and `update_or_create()`

```
product, created = Product.objects.get_or_create(
    name="Notebook",
    defaults={"price": "100.00"},
)
```

```
product, created = Product.objects.update_or_create(
    name="Notebook",
    defaults={"price": "120.00"},
)
```

Race condition এড়াতে উপযুক্ত database uniqueness constraints থাকা গুরুত্বপূর্ণ।

10.13 Raw SQL

সাধারণত ORM-কে অগ্রাধিকার দাও। Raw SQL দরকার হলে parameters ব্যবহার করো।

```
Product.objects.raw(
    "SELECT * FROM core_product WHERE id = %s",
    [1],
)
```

User input দিয়ে SQL string concatenate করো না।

Chapter 11: CRUD Operations

CRUD অর্থ Create, Read, Update এবং Delete।

11.1 Create

```
product = Product.objects.create(
    name="Book",
    price="250.00",
)
```

11.2 Read

```
products = Product.objects.all()
```

11.3 Update

```
product = Product.objects.get(pk=1)
product.price = "300.00"
product.save()
```

11.4 Delete

```
product = Product.objects.get(pk=1)
product.delete()
```

11.5 Safe Detail View

```
from django.shortcuts import get_object_or_404

product = get_object_or_404(Product, pk=product_id)
```

11.6 CRUD Best Practices

- Form বা serializer দিয়ে validate করো।
- Login এবং permissions যাচাই করো।
- Missing objects-এর জন্য 404 handle করো।
- Delete action POST বা উপযুক্ত non-GET method দিয়ে করো।
- Success/failure messages দিতে পারো।
- Automated tests লেখো।

Chapter 12: Forms and ModelForms

12.1 Django Form

`forms.py`:

```
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
```

12.2 Form View

```
from django.shortcuts import redirect, render
from .forms import ContactForm

def contact(request):
    if request.method == "POST":
        form = ContactForm(request.POST)

        if form.is_valid():
```

```
            name = form.cleaned_data["name"]
            email = form.cleaned_data["email"]
            message = form.cleaned_data["message"]

            # Process validated data.
            return redirect("home")
    else:
        form = ContactForm()

    return render(request, "contact.html", {"form": form})
```

Template:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
</form>
```

12.3 ModelForm

```
from django import forms
from .models import Product

class ProductForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = ["name", "price", "description"]
```

12.4 Custom Validation

```
def clean_name(self):
    name = self.cleaned_data["name"]

    if len(name.strip()) < 3:
        raise forms.ValidationError(
            "Name must contain at least 3 characters."
        )

    return name
```

Cross-field validation:

```
def clean(self):
    cleaned_data = super().clean()
    # Validate related fields here.
    return cleaned_data
```

12.5 `save(commit=False)`

```
product = form.save(commit=False)
product.created_by = request.user
product.save()
form.save_m2m()
```

`save_m2m()` দরকার হতে পারে যদি form-এ many-to-many fields থাকে এবং প্রথমে `commit=False` ব্যবহার করা হয়।

12.6 Formsets and Inline Formsets

- **Formset:** একসঙ্গে একাধিক একই ধরনের form পরিচালনা করে।
- **Inline formset:** Parent object-এর সঙ্গে related child objects edit করতে সাহায্য করে।

উদাহরণ: একটি order-এর একাধিক order item।

12.7 Form Security

- CSRF token ব্যবহার করো।
- Sensitive fields form-এ expose করো না।
- Server-side validation বজায় রাখো।
- `fields = "__all__"` অন্ধভাবে ব্যবহার করো না।

Chapter 13: Django Admin

Admin interface দিয়ে database records manage করা যায়।

`admin.py`:

```
from django.contrib import admin
from .models import Product

@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "price", "is_active")
    list_filter = ("is_active",)
    search_fields = ("name", "description")
    ordering = ("name",)
    list_per_page = 25
```

Superuser:

```
python manage.py createsuperuser
```

Admin URL সাধারণত `/admin/`।

Advanced admin features:

- `readonly_fields`
- `fieldsets`
- `list_editable`
- `autocomplete_fields`
- Inline models
- Custom admin actions
- Permission restrictions

Admin access শুধু authorized users-কে দাও এবং production configuration নিরাপদ রাখো।

Chapter 14: Authentication and Authorization

14.1 Authentication vs Authorization

- **Authentication:** ব্যবহারকারী কে?
- **Authorization:** ব্যবহারকারী কী করতে পারবে?

14.2 Login and Logout

Django built-in authentication views ও forms ব্যবহার করা যায়। Custom implementation প্রয়োজন হলে Django-এর authentication utilities ব্যবহার করো; password নিজে compare বা plain-text-এ store করো না।

```
from django.contrib.auth import authenticate, login, logout
from django.shortcuts import redirect, render

def login_view(request):
    if request.method == "POST":
        username = request.POST.get("username", "")
        password = request.POST.get("password", "")

        user = authenticate(
            request,
            username=username,
            password=password,
        )
```

```
        if user is not None:
            login(request, user)
            return redirect("home")

    return render(request, "login.html")
```

Logout:

```
def logout_view(request):
    logout(request)
    return redirect("home")
```

Logout action-টি POST form-এর মাধ্যমে implement করা ভালো।

14.3 Protect a View

```
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

14.4 Permissions

```
from django.contrib.auth.decorators import permission_required

@permission_required("core.change_product", raise_exception=True)
def edit_product(request):
    ...
```

Django default model permissions সাধারণত add, change, delete এবং view-এর জন্য তৈরি করে।

14.5 Groups

Groups দিয়ে একাধিক user-কে common permissions দেওয়া যায়।

উদাহরণ:

- Managers
- Editors
- Support staff

14.6 Custom User Model

Custom user model প্রয়োজন হলে project-এর শুরুতেই সেটি নির্ধারণ করা উত্তম।

```
AUTH_USER_MODEL = "accounts.CustomUser"
```

Models-এ user relationship-এর জন্য:

```
from django.conf import settings

user = models.ForeignKey(
    settings.AUTH_USER_MODEL,
    on_delete=models.CASCADE,
)
```

14.7 Password Reset and Verification

Django authentication tools password change/reset workflow তৈরি করতে সাহায্য করে। Email delivery, token expiry এবং account verification flow নিরাপদভাবে configure করতে হবে।

14.8 Object-Level Permissions

একজন user নিজের product edit করতে পারবে, অন্যের product নয়—এ ধরনের rules view বা permission layer-এ enforce করতে হবে।

Chapter 15: Sessions and Cookies

15.1 Sessions

Sessions user-specific state server-side storage-এ রাখতে সাহায্য করে।

```
request.session["cart_count"] = 3
count = request.session.get("cart_count", 0)
request.session.pop("cart_count", None)
```

Session configuration `SESSION_ENGINE` এবং related settings দিয়ে নিয়ন্ত্রণ করা যায়।

15.2 Cookies

```
from django.http import HttpResponse

def set_cookie_view(request):
    response = HttpResponse("Cookie set")
    response.set_cookie(
        "theme",
        "dark",
        max_age=3600,
        httponly=True,
        samesite="Lax",
        secure=True,
    )
    return response
```

- `HttpOnly` : JavaScript access সীমিত করে।
- `Secure` : HTTPS connection-এ cookie পাঠায়।
- `SameSite` : Cross-site request-এর সঙ্গে cookie পাঠানো নিয়ন্ত্রণে সাহায্য করে।

Cookies-এ password বা secret সরাসরি রাখবে না।

Chapter 16: Middleware

16.1 What Is Middleware?

Middleware হলো request এবং response processing-এর মাঝখানে কাজ করা component।

Common middleware:

- `SecurityMiddleware`
- `SessionMiddleware`
- `CommonMiddleware`
- `CsrfViewMiddleware`
- `AuthenticationMiddleware`
- `MessageMiddleware`

16.2 Custom Middleware

```
class SimpleMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # Before view
        response = self.get_response(request)
        # After view
        return response
```

`settings.py`-এর `MIDDLEWARE` list-এ middleware class যোগ করতে হয়।

16.3 Middleware Order

Middleware order গুরুত্বপূর্ণ। যেমন, authentication middleware সাধারণত session middleware-এর পরে থাকে, কারণ authentication user resolve করতে session ব্যবহার করতে পারে।

16.4 Middleware Uses

- Request logging
- Security headers
- Request IDs
- Custom authentication behavior

- Response modification
- Maintenance-mode handling

Middleware-এ expensive work অপ্রয়োজনীয়ভাবে চালাবে না।

Chapter 17: Model Relationships

17.1 ForeignKey — Many-to-One

```
class Category(models.Model):
    name = models.CharField(max_length=100)

class Product(models.Model):
    category = models.ForeignKey(
        Category,
        on_delete=models.PROTECT,
        related_name="products",
    )
    name = models.CharField(max_length=200)
```

একটি category-তে অনেক product থাকতে পারে।

17.2 OneToOneField

```
from django.conf import settings
from django.db import models

class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
    )
    bio = models.TextField(blank=True)
```

17.3 ManyToManyField

```
class Student(models.Model):
    name = models.CharField(max_length=100)

class Course(models.Model):
    title = models.CharField(max_length=100)
    students = models.ManyToManyField(
        Student,
```

```
        related_name="courses",
    )
```

17.4 Through Model

Relationship-এর সঙ্গে extra data রাখতে intermediate model ব্যবহার করা যায়।

```
class Membership(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    enrolled_at = models.DateTimeField(auto_now_add=True)
```

Many-to-many field-এ `through="Membership"` configure করে relationship data manage করা যায়।

17.5 `on_delete`

- `CASCADE` : Related object delete হলে dependent object-ও delete হয়।
- `PROTECT` : Delete আটকায়।
- `SET_NULL` : Nullable relation-কে NULL করে।
- `SET_DEFAULT` : Default value ব্যবহার করে।
- `RESTRICT` : নির্দিষ্ট related-object constraints অনুযায়ী deletion restrict করে।

Chapter 18: Query Optimization

18.1 N+1 Query Problem

একটি list-এর প্রতিটি object-এর related data আলাদাভাবে fetch করলে query সংখ্যা বেড়ে যেতে পারে।

18.2 `select_related()`

ForeignKey এবং OneToOne relationships-এর জন্য JOIN ব্যবহার করে।

```
products = Product.objects.select_related("category")
```

18.3 `prefetch_related()`

Many-to-many এবং reverse relationships-এর জন্য related data আলাদা query করে যুক্ত করে।

```
courses = Course.objects.prefetch_related("students")
```

18.4 Limit Fields

```
Product.objects.only("id", "name")
```

```
Product.objects.defer("description")
```

Deferred field পরে access করলে অতিরিক্ত query হতে পারে। Profile actual access pattern দেখে optimize করো।

18.5 Indexes

```
sku = models.CharField(max_length=50, db_index=True)
```

Index read query দ্রুত করতে পারে, তবে write overhead এবং storage বাড়াতে পারে।

18.6 Bulk Operations

```
Product.objects.bulk_create([
    Product(name="Book", price="200.00"),
    Product(name="Pen", price="20.00"),
])
```

Bulk operations সাধারণ `save()` এবং signals-এর মতো একই lifecycle অনুসরণ করে না।

18.7 Explain Query

```
print(Product.objects.filter(is_active=True).explain())
```

Database query plan বুঝতে সাহায্য করে। Database backend অনুযায়ী output আলাদা হতে পারে।

Chapter 19: Django REST Framework (DRF)

19.1 What Is DRF?

Django REST Framework ব্যবহার করে REST API তৈরি করা যায়।

Features:

- Serializers
- Validation
- API views
- ViewSets and routers
- Authentication
- Permissions
- Pagination
- Filtering
- Browsable API
- API testing tools

19.2 Install

```
python -m pip install djangorestframework
```

`settings.py`:

```
INSTALLED_APPS = [
    # Existing apps
    "rest_framework",
    "core",
]
```

19.3 Simple API View

```
from rest_framework.decorators import api_view
from rest_framework.response import Response

@api_view(["GET"])
def health_check(request):
    return Response({"status": "ok"})
```

URL:

```
path("api/health/", views.health_check, name="health_check")
```

19.4 JSON Response

```
{
  "id": 1,
  "name": "Keyboard",
  "price": "1500.00"
}
```

API-তে consistent response structure, validation এবং correct status codes থাকা দরকার।

Chapter 20: Serializers and Validation

20.1 ModelSerializer

`serializers.py`:

```
from rest_framework import serializers
from .models import Product

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
```

```
        fields = ["id", "name", "price", "description"]
        read_only_fields = ["id"]
```

20.2 Field Validation

```
class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ["id", "name", "price"]

    def validate_price(self, value):
        if value <= 0:
            raise serializers.ValidationError(
                "Price must be greater than zero."
            )
        return value
```

20.3 Object-Level Validation

```
def validate(self, attrs):
    # Validate relationships between fields.
    return attrs
```

20.4 Serializer Save

```
serializer = ProductSerializer(data=request.data)

if serializer.is_valid():
    product = serializer.save()
```

Invalid input:

```
serializer.is_valid(raise_exception=True)
```

20.5 Create and Update

Custom `create()` বা `update()` methods লিখতে হয় যখন default `ModelSerializer` behavior যথেষ্ট নয়।

20.6 Read-Only and Write-Only Fields

- `read_only=True` : Output-এ field থাকবে, input থেকে পরিবর্তন করা যাবে না।
- `write_only=True` : Input-এ ব্যবহার হবে, output-এ দেখানো হবে না।

Sensitive fields-এর ক্ষেত্রে উপযুক্ত field configuration জরুরি।

Chapter 21: API Views, ViewSets and Routers

21.1 APIView

```
from rest_framework.views import APIView
from rest_framework.response import Response

class HealthAPIView(APIView):
    def get(self, request):
        return Response({"status": "ok"})
```

21.2 GenericAPIView and Mixins

Generic views common CRUD behavior reuse করতে সাহায্য করে। Mixins দিয়ে list, create, retrieve, update ও destroy actions combine করা যায়।

21.3 ModelViewSet

```
from rest_framework.viewsets import ModelViewSet
from .models import Product
from .serializers import ProductSerializer

class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

21.4 Router

```
from rest_framework.routers import DefaultRouter
from .views import ProductViewSet

router = DefaultRouter()
router.register("products", ProductViewSet, basename="product")

urlpatterns = router.urls
```

`ModelViewSet` সাধারণত list, retrieve, create, update, partial update এবং destroy actions দেয়।

21.5 Limit Querysets

```
def get_queryset(self):
    return Product.objects.filter(is_active=True)
```

Multi-user application-এ current user-এর অনুমোদিত records-এ queryset সীমাবদ্ধ করা গুরুত্বপূর্ণ।

21.6 API Versioning

API versioning client compatibility বজায় রাখতে সাহায্য করে। URL, namespace বা configured versioning scheme ব্যবহার করা যেতে পারে।

21.7 API Documentation

OpenAPI schema এবং Swagger/ReDoc-এর মতো tools দিয়ে API documentation তৈরি করা যায়। Documentation-এ authentication এবং request/response schemas স্পষ্টভাবে দেখানো উচিত।

Chapter 22: API Authentication and Permissions

22.1 Authentication Methods

- Session authentication
- Basic authentication
- Token authentication
- JWT authentication
- OAuth2-based authentication

কোন পদ্ধতি ব্যবহার হবে তা client type, threat model এবং deployment requirements-এর ওপর নির্ভর করে।

22.2 DRF Permissions

```
from rest_framework.permissions import IsAuthenticated
from rest_framework.viewsets import ModelViewSet

class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticated]
```

22.3 Read-Only Access

```
from rest_framework.permissions import IsAuthenticatedOrReadOnly
```

22.4 Object-Level Permissions

নিজের object edit করতে পারবে কিন্তু অন্য user-এর object নয়—এ ধরনের rules enforce করতে object-level permissions এবং scoped querysets ব্যবহার করো।

22.5 JWT

JWT-এর জন্য maintained package ব্যবহার করো এবং access token expiry, refresh, revocation strategy, signing secret এবং secure storage বিবেচনা করো।

22.6 Throttling

DRF throttling API usage limit করতে সাহায্য করে। এটি সব ধরনের malicious traffic প্রতিরোধের বিকল্প নয়; প্রয়োজন অনুযায়ী reverse proxy বা gateway-level controls-ও লাগতে পারে।

22.7 CORS and CSRF

- **CORS:** Browser কোন origin থেকে cross-origin response access করতে পারবে তা নিয়ন্ত্রণ করে।
- **CSRF:** Browser-এর authenticated session ব্যবহার করে অনিচ্ছাকৃত state-changing request পাঠানোর ঝুঁকি মোকাবিলা করে।

দুটি আলাদা security mechanism।

Chapter 23: Filtering, Searching, Ordering and Pagination

23.1 Filtering

Install:

```
python -m pip install django-filter
```

Configuration:

```
INSTALLED_APPS = [
    # Existing apps
    "django_filters",
]
```

ViewSet:

```
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.viewsets import ModelViewSet

class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    filter_backends = [DjangoFilterBackend]
    filterset_fields = ["category", "is_active"]
```

23.2 Searching

```
from rest_framework.filters import SearchFilter
```

```
filter_backends = [SearchFilter]
search_fields = ["name", "description"]
```

23.3 Ordering

```
from rest_framework.filters import OrderingFilter

filter_backends = [OrderingFilter]
ordering_fields = ["name", "price"]
ordering = ["name"]
```

শুধু অনুমোদিত ordering fields expose করো।

23.4 Pagination

```
REST_FRAMEWORK = {
    "DEFAULT_PAGINATION_CLASS":
        "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 10,
}
```

Pagination types:

- Page-number pagination
- Limit-offset pagination
- Cursor pagination

বড় dataset-এ cursor pagination কিছু use case-এ উপযোগী হতে পারে।

Chapter 24: Signals, Custom Managers and Commands

24.1 Signals

Signal নির্দিষ্ট event ঘটলে receiver function চালায়।

```
from django.db.models.signals import post_save
from django.dispatch import receiver

@receiver(post_save, sender=Product)
def product_saved(sender, instance, created, **kwargs):
    if created:
        # Handle new product.
        pass
```

Receiver import হওয়ার ব্যবস্থা নিশ্চিত করতে হবে, সাধারণত app configuration-এর মাধ্যমে।

Signals অতিরিক্ত ব্যবহার করলে control flow বোঝা কঠিন হয়। Explicit service function অনেক ক্ষেত্রে পরিষ্কার সমাধান।

24.2 Custom Manager

```
class ActiveProductManager(models.Manager):
    def get_queryset(self):
        return super().get_queryset().filter(is_active=True)
```

Model-এ manager যোগ করে reusable query logic তৈরি করা যায়।

24.3 Custom QuerySet

```
class ProductQuerySet(models.QuerySet):
    def active(self):
        return self.filter(is_active=True)
```

Custom QuerySet methods chainable query behavior তৈরি করতে সাহায্য করে।

24.4 Management Commands

Custom management command দিয়ে imports, reports, cleanup এবং maintenance automate করা যায়।

Typical structure:

```
core/
└── management/
    └── commands/
        └── import_products.py
```

Command class-এ `BaseCommand` extend করে `handle()` implement করা হয়।

Chapter 25: Error Handling and Logging

25.1 Common Exceptions

- `DoesNotExist`
- `MultipleObjectsReturned`
- `ValidationError`
- `PermissionDenied`
- `Http404`
- Database exceptions

25.2 Handle Missing Object

```
from django.shortcuts import get_object_or_404

product = get_object_or_404(Product, pk=product_id)
```

25.3 Logging

```
import logging

logger = logging.getLogger(__name__)

def process_data():
    try:
        # Perform operation.
        pass
    except ValueError:
        logger.exception("Data processing failed")
        raise
```

25.4 Logging Levels

- `DEBUG`
- `INFO`
- `WARNING`
- `ERROR`
- `CRITICAL`

Production logs-এ password, tokens বা sensitive personal data রাখবে না।

25.5 Custom Error Pages

Django custom 404 এবং 500 templates support করে। Production-এ `DEBUG=False` রাখো এবং errors পর্যবেক্ষণ করার ব্যবস্থা রাখো।

Chapter 26: Testing and Debugging

26.1 TestCase

```
from django.test import TestCase
from .models import Product

class ProductTests(TestCase):
    def test_product_creation(self):
        product = Product.objects.create(
            name="Book",
            price="250.00",
```

```
        )
        self.assertEqual(product.name, "Book")
```

Run:

```
python manage.py test
```

26.2 Test Client

```
from django.test import TestCase
from django.urls import reverse

class HomeTests(TestCase):
    def test_home(self):
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)
```

26.3 API Testing

DRF-এর `APITestCase` এবং `APIClient` ব্যবহার করে API request, permissions, validation এবং response পরীক্ষা করা যায়।

26.4 Test Coverage

Coverage tools দিয়ে কোন code test হয়েছে তা পরিমাপ করা যায়। High coverage থাকলেই সব behavior সঠিক—এমন নয়; meaningful assertions দরকার।

26.5 What to Test

- Model behavior
- Form validation
- URL resolution
- View response
- Authentication
- Authorization
- Serializer validation
- API status codes
- Error cases
- Transactions
- Regression scenarios

26.6 Debugging Checklist

1. Error message এবং traceback পড়ো।

2. `python manage.py check` চালাও।
3. Settings এবং environment যাচাই করো।
4. URL pattern এবং view inspect করো।
5. Database migration status দেখো।
6. Query count এবং logs পরীক্ষা করো।
7. সমস্যাটি reproduce করার test লেখো।

Chapter 27: Django Security

27.1 DEBUG

Production-এ:

```
DEBUG = False
```

27.2 Secret Key

Secret environment variable বা secret manager থেকে load করো। Production secret GitHub-এ commit করো না।

27.3 Allowed Hosts

```
ALLOWED_HOSTS = [
    "example.com",
    "www.example.com",
]
```

নিজের domain ব্যবহার করবে।

27.4 CSRF

HTML POST forms-এ:

```
{% csrf_token %}
```

CSRF protection অকারণে disable করো না।

27.5 SQL Injection

Django ORM parameterized queries ব্যবহার করে। Raw SQL-এ user input concatenate করো না।

27.6 XSS

Template autoescaping বজায় রাখো। Untrusted content-এর জন্য অপ্রয়োজনীয়ভাবে `safe` filter ব্যবহার করো না।

27.7 Password Security

- Django password hashing ব্যবহার করো।
- Plain-text password store করো না।
- Secure password reset workflow রাখো।
- প্রয়োজন অনুযায়ী login rate limiting যোগ করো।

27.8 HTTPS and Cookies

Production-এ HTTPS, secure cookies এবং উপযুক্ত security headers configure করো।

27.9 File Upload Security

- File size limit রাখো।
- File type ও content validate করো।
- User-uploaded files-কে trusted executable content হিসেবে বিবেচনা করো না।
- Private files-এর access control রাখো।
- প্রয়োজন হলে isolated storage ব্যবহার করো।

27.10 Deployment Security Check

```
python manage.py check --deploy
```

Warnings review করো, dependencies update রাখো এবং production settings পরীক্ষা করো।

Chapter 28: Caching

28.1 What Is Caching?

Repeated computation বা database access কমাতে data সাময়িকভাবে cache করা হয়।

28.2 Cache API

```
from django.core.cache import cache

cache.set("homepage_title", "Welcome", timeout=300)
title = cache.get("homepage_title")
cache.delete("homepage_title")
```

28.3 Cache Types

- Per-site cache
- Per-view cache
- Template fragment cache
- Low-level cache API

28.4 Cache Backends

Production-এ Redis বা Memcached ব্যবহার করা যেতে পারে। Cache configuration environment এবং workload অনুযায়ী নির্ধারণ করতে হবে।

28.5 Cache Invalidation

Data update হলে stale cache কীভাবে expire বা invalidate হবে তা পরিকল্পনা করো। Sensitive user-specific data cache করলে isolation এবং access control বিবেচনা করো।

Chapter 29: Background Tasks with Celery

29.1 Why Background Tasks?

দীর্ঘ সময় লাগে এমন কাজ web request থেকে আলাদা করলে response দ্রুত হতে পারে।

উদাহরণ:

- Email delivery
- Report generation
- Scheduled cleanup
- Data import
- External API processing

29.2 Architecture

```
Django
   ↓
Message Broker
   ↓
Celery Worker
   ↓
Task Execution
```

29.3 Example Task

```
from celery import shared_task

@shared_task
def add_numbers(a, b):
    return a + b
```

Call:

```
add_numbers.delay(10, 20)
```

29.4 Production Considerations

- Redis বা RabbitMQ broker
- Worker process management
- Task retries
- Idempotency
- Timeouts
- Failure monitoring
- Scheduled-task configuration

Task retry করলে একই operation একাধিকবার execute হতে পারে—তাই task behavior পরিকল্পনা করা জরুরি।

Chapter 30: Email Integration

30.1 Send Email

```
from django.core.mail import send_mail

send_mail(
    subject="Welcome",
    message="Welcome to our website.",
    from_email=None,
    recipient_list=["user@example.com"],
    fail_silently=False,
)
```

30.2 Common Use Cases

- Password reset
- Email verification
- Order confirmation
- Notifications
- Contact forms
- Scheduled reports

30.3 Production Email

- Email provider configure করো।
- Credentials environment variables-এ রাখো।
- Delivery failures log করো।
- Rate limits এবং spam prevention বিবেচনা করো।

- Bulk email-এর জন্য background tasks ব্যবহার করতে পারো।

Chapter 31: File Uploads and Storage

31.1 Model

```
class Document(models.Model):
    title = models.CharField(max_length=200)
    file = models.FileField(upload_to="documents/")
```

31.2 Upload Form

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Upload</button>
</form>
```

31.3 View

```
form = DocumentForm(
    request.POST,
    request.FILES,
)
```

Valid form হলে `form.save()` ব্যবহার করা যায়।

31.4 ImageField

```
image = models.ImageField(upload_to="products/")
```

ImageField-এর জন্য উপযুক্ত image-processing dependency প্রয়োজন হতে পারে।

31.5 Storage Backends

- Local filesystem
- S3-compatible object storage
- Managed cloud storage

File storage location এবং database backup strategy আলাদাভাবে পরিকল্পনা করো।

Chapter 32: Environment Variables and Configuration

32.1 Environment Variables

```
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]

DEBUG = (
    os.environ.get("DJANGO_DEBUG", "False").lower() == "true"
)
```

Required secret না থাকলে application fail fast করা অনেক সময় নিরাপদ।

32.2 Database Configuration

Production database credentials environment variables বা secure secret manager থেকে load করো।

32.3 Settings Modules

```
config/
└── settings/
    ├── __init__.py
    ├── base.py
    ├── development.py
    └── production.py
```

একাধিক settings module ব্যবহার করলে `DJANGO_SETTINGS_MODULE` সঠিকভাবে configure করতে হবে।

32.4 Configuration Best Practices

- Secrets source code-এ রাখবে না।
- Development ও production settings আলাদা রাখো।
- Environment variable validation করো।
- Production settings test করো।
- `.env` file ব্যবহার করলে সেটি Git-এ commit করো না।

Chapter 33: Git and GitHub Workflow

33.1 Common Commands

```
git init
git status
git add .
git commit -m "Add product model"
git branch
```

```
git pull
git push
```

Remote repository:

```
git remote add origin <repository-url>
git push -u origin main
```

33.2 `.gitignore`

```
.venv/
__pycache__/
*.py[cod]
.env
db.sqlite3
media/
staticfiles/
```

Project-এর প্রয়োজন অনুযায়ী ignore rules পরিবর্তন করবে। Migration files সাধারণত version control-এ রাখা উচিত।

33.3 Requirements

```
python -m pip freeze > requirements.txt
```

এই command current environment-এর packages freeze করে। Production project-এ dependencies review, pin এবং update করা উচিত।

33.4 Git Workflow

1. Feature branch তৈরি করো।
2. ছোট ও পরিষ্কার commits করো।
3. Tests চালাও।
4. Pull request তৈরি করো।
5. Code review করো।
6. Merge এবং deploy করো।

33.5 README

README-তে project description, setup, environment variables, migrations, tests, API endpoints এবং deployment instructions রাখো।

Chapter 34: Production Deployment

34.1 Production Checklist

1. `DEBUG=False`
2. Correct `ALLOWED_HOSTS`
3. Secret key protected
4. Production database configured
5. HTTPS enabled
6. Static files collected
7. Media storage configured
8. Migrations applied
9. Logging configured
10. Database backups arranged
11. Dependencies updated
12. Deployment checks passed

34.2 Collect Static

```
python manage.py collectstatic
```

34.3 Apply Migrations

```
python manage.py migrate
```

34.4 WSGI vs ASGI

- **WSGI:** Traditional synchronous web application interface।
- **ASGI:** Async-compatible application interface, including supported long-lived connections।

34.5 Production Server

Django development server-এর পরিবর্তে উপযুক্ত production application server ব্যবহার করো।

Deployment platform অনুযায়ী process management এবং reverse proxy configure করতে হয়।

34.6 Production Database

PostgreSQL একটি common production choice। Database user permissions, backups, connection settings এবং migrations পরিকল্পনা করো।

34.7 Monitoring

- Error logging
- Health checks
- CPU/memory monitoring

- Database metrics
- Application latency
- Backup verification
- Dependency/security updates

Deployment platform-এর documentation অনুযায়ী final configuration নির্ধারণ করবে।

Chapter 35: Performance Optimization

35.1 Optimization Process

1. Slow endpoints identify করো।
2. Database queries inspect করো।
3. N+1 queries fix করো।
4. Appropriate indexes যোগ করো।
5. Pagination ব্যবহার করো।
6. Caching বিবেচনা করো।
7. Background tasks ব্যবহার করো।
8. Load testing করো।
9. Metrics monitor করো।

35.2 Database Transactions

```
from django.db import transaction

with transaction.atomic():
    # Related database operations
    pass
```

একটি operation fail হলে transaction-এর database changes rollback হতে পারে।

35.3 `select_for_update()`

Concurrent updates-এর কিছু ক্ষেত্রে row locking প্রয়োজন হতে পারে।

```
from django.db import transaction

with transaction.atomic():
    product = Product.objects.select_for_update().get(pk=1)
    # Perform coordinated update.
```

এটির behavior database backend এবং transaction configuration-এর ওপর নির্ভর করে।

35.4 Avoid Premature Optimization

প্রথমে profiling এবং measurements করো। অনুমান করে caching বা indexes যোগ না করে actual bottleneck identify করো।

Chapter 36: Docker and CI/CD

36.1 Docker Concepts

- Image
- Container
- Dockerfile
- Docker Compose
- Volumes
- Networks
- Environment variables

Docker development এবং deployment environments consistent রাখতে সাহায্য করে।

36.2 Dockerfile Outline

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["gunicorn", "config.wsgi:application"]
```

এটি একটি starting example; production-এর জন্য port binding, non-root user, static files, health checks, secrets এবং process configuration অনুযায়ী update করতে হবে। `gunicorn` dependency install করতে হবে।

36.3 Docker Compose

Compose দিয়ে Django, PostgreSQL এবং Redis-এর মতো services development environment-এ একসঙ্গে চালানো যায়।

36.4 CI/CD Pipeline

```
Git Push
   ↓
Automated Tests
   ↓
Lint and Checks
   ↓
Build Image
   ↓
Deploy
   ↓
Health Check
```

36.5 CI/CD Best Practices

- Secrets secure রাখো।
- Tests pass না করলে deploy আটকাও।
- Migration strategy আগে নির্ধারণ করো।
- Rollback strategy রাখো।
- Production deployment-এ health checks ব্যবহার করো।

Chapter 37: Advanced Django Concepts

37.1 Abstract Models

Shared fields একাধিক model-এ reuse করতে abstract base model ব্যবহার করা যায়।

```
class TimeStampedModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True
```

37.2 Custom Model Methods

```
class Product(models.Model):
    name = models.CharField(max_length=200)

    def __str__(self):
        return self.name
```

Model-এর সঙ্গে সম্পর্কিত behavior model method-এ রাখা যায়।

37.3 Custom Fields

Custom field তৈরি করা সম্ভব, তবে built-in field যথেষ্ট না হলে তবেই ব্যবহার করো। Database conversion এবং validation behavior সঠিকভাবে implement করতে হয়।

37.4 App Configuration

`apps.py`-তে `AppConfig` app metadata এবং startup configuration পরিচালনা করে। App initialization-এ database query বা expensive work অপ্রয়োজনীয়ভাবে চালাবে না।

37.5 Async Django

Django async views সমর্থন করে।

```
from django.http import JsonResponse

async def async_health_check(request):
    return JsonResponse({"status": "ok"})
```

Async code-এ synchronous database operations এবং third-party libraries-এর compatibility যাচাই করতে হবে।

37.6 Internationalization

Django translation ও localization support করে। Language, locale, timezone এবং translated strings project requirements অনুযায়ী configure করো।

37.7 Reusable Apps

Reusable app-এর মধ্যে পরিষ্কার models, URLs, migrations, templates, tests এবং dependencies রাখা উচিত। App-specific configuration documentation-এ উল্লেখ করো।

37.8 Custom Permissions

Model-এর `Meta` class-এ custom permissions define করা যায় এবং পরে user/group permissions দিয়ে enforce করা যায়।

37.9 Transactions and Concurrency

Transactions, unique constraints এবং প্রয়োজনে row-level locking concurrent updates-এর consistency বজায় রাখতে সাহায্য করে।

37.10 Architecture and Service Layer

Project বড় হলে business operations reusable service functions-এ organize করা যায়। তবে অপ্রয়োজনীয় abstraction তৈরি না করে codebase-এর complexity অনুযায়ী architecture নির্ধারণ করো।

Chapter 38: Practical Projects

38.1 Beginner Projects

Project 1: To-Do Application

Features:

- Create task
- List tasks
- Update task
- Delete task
- Mark complete
- Form validation

Project 2: Blog Application

Features:

- Post CRUD
- Categories
- Search
- Pagination
- Admin panel

Project 3: Portfolio Website

Features:

- About page
- Skills
- Projects
- Contact form
- Static files
- Deployment

38.2 Intermediate Projects

Project 4: Inventory Management

Features:

- Product and category models
- Stock quantity
- Suppliers
- Stock movement
- Search and filters

- Authentication
- Reports

Project 5: Library Management

Features:

- Books
- Members
- Borrow and return
- Due dates
- Search
- Admin dashboard

Project 6: Expense Tracker

Features:

- Authentication
- Income and expenses
- Categories
- Monthly reports
- Pagination
- Charts

38.3 Advanced Projects

Project 7: E-commerce Backend

Features:

- Products and categories
- Cart
- Orders
- Inventory validation
- User authentication
- Payment provider integration
- Order status
- Email notifications
- Background tasks
- API testing

Project 8: Job Portal API

Features:

- Candidate and employer accounts
- Job listings
- Applications
- Permissions
- Filtering and pagination
- Resume uploads
- Email notifications

Project 9: Appointment Booking API

Features:

- User accounts
- Availability
- Booking conflicts
- Transaction handling
- Notifications
- Cancellation rules
- Automated tests

38.4 Project Quality Checklist

প্রতিটি project-এ চেষ্টা করবে:

- Clean project structure
- Appropriate models and relationships
- Input validation
- Authentication and authorization
- Error handling
- Automated tests
- Security configuration
- Git history
- README
- API documentation where applicable
- Deployment

Chapter 39: Interview Questions

Q1. What is Django?

Django হলো Python-based high-level web framework, যা secure এবং maintainable web application তৈরি করতে সাহায্য করে।

Q2. What is MVT?

Model data layer, View request-processing logic এবং Template presentation layer পরিচালনা করে।

Q3. What is ORM?

ORM Python objects এবং database records-এর মধ্যে mapping তৈরি করে।

Q4. What are migrations?

Model schema changes database-এ প্রয়োগ করার versioned mechanism।

Q5. What is middleware?

Request ও response processing-এর মধ্যে কাজ করা component।

Q6. What is the difference between `null` and `blank`?

`null` database-level value নিয়ে কাজ করে; `blank` validation-এ empty input অনুমোদন করে।

Q7. What is `select_related()`?

ForeignKey ও OneToOne relationships-এর জন্য JOIN-based query optimization।

Q8. What is `prefetch_related()`?

Related records আলাদা query করে Python-এ যুক্ত করে।

Q9. What is a QuerySet?

Database query represent করা object, যা সাধারণত lazy evaluation ব্যবহার করে।

Q10. What is the difference between FBV and CBV?

FBV function দিয়ে তৈরি; CBV class ও methods দিয়ে তৈরি। CBV inheritance এবং generic views দিয়ে reusable behavior দিতে পারে।

Q11. What is `get_object_or_404()`?

Object পাওয়া না গেলে `Http404` raise করে।

Q12. What is CSRF?

Cross-Site Request Forgery attack-এ authenticated browser-কে অনিচ্ছাকৃত request পাঠাতে প্ররোচিত করা হতে পারে।

Q13. What is Django REST Framework?

Django-তে API তৈরি, serialization, validation, authentication এবং permissions পরিচালনার framework।

Q14. What is a serializer?

Data representation এবং validation পরিচালনা করে।

Q15. Authentication vs authorization?

Authentication পরিচয় যাচাই করে; authorization access অনুমোদন করে।

Q16. What is a ViewSet?

DRF class যা একাধিক related API actions একত্রে পরিচালনা করতে পারে।

Q17. What is pagination?

Large dataset-কে ছোট ছোট response pages-এ ভাগ করা।

Q18. What is caching?

Repeated data retrieval বা computation-এর খরচ কমাতে temporary storage ব্যবহার করা।

Q19. What is Celery?

Background এবং scheduled tasks চালানোর task queue system।

Q20. What is `transaction.atomic()`?

একাধিক database operation-কে atomic transaction হিসেবে execute করতে সাহায্য করে।

Q21. How do you secure a Django application?

Secure settings, validation, authentication, permissions, CSRF protection, HTTPS, dependency updates এবং safe file handling ব্যবহার করে।

Q22. How do you improve Django performance?

Query optimization, indexes, pagination, caching, profiling এবং appropriate background tasks ব্যবহার করে।

Q23. What is a custom user model?

Django user behavior বা fields project requirements অনুযায়ী customize করার model।

Q24. What is the purpose of `on_delete`?

ForeignKey বা OneToOne relation-এর referenced object delete হলে dependent record-এর behavior নির্ধারণ করে।

Q25. What is Docker?

Application এবং তার dependencies-কে containerized environment-এ চালানোর platform।

Q26. What is CI/CD?

Continuous Integration এবং Continuous Delivery/Deployment automated testing, build এবং release workflow-কে সংগঠিত করে।

Q27. How do you deploy Django?

Production settings, database, static/media storage, production server, HTTPS, migrations, logging এবং monitoring configure করে।

Chapter 40: Final A-to-Z Checklist

Python Fundamentals

- Variables and data types
- Conditions and loops
- Functions
- Collections
- OOP
- Exceptions
- Modules and packages
- Virtual environments
- Basic SQL and HTTP

Core Django

- Project and app creation
- Settings
- URL routing
- URL namespaces and reversing
- Function-based views
- Class-based views
- Generic views
- View decorators and mixins
- Templates and inheritance
- Static and media files
- Models and migrations
- ORM and CRUD
- Forms and ModelForms
- Form validation
- Admin customization
- Authentication

- Authorization and permissions
- Sessions and cookies
- Middleware
- Model relationships
- Query optimization

Django REST Framework

- REST fundamentals
- Serializers
- Field and object validation
- APIView
- GenericAPIView
- Mixins
- ViewSets and routers
- Authentication
- Object-level permissions
- Filtering
- Searching
- Ordering
- Pagination
- Throttling
- API versioning
- API documentation
- API testing

Advanced Django

- Signals
- Custom managers and QuerySets
- Management commands
- Error handling
- Logging
- Automated tests
- Security checks
- Transactions
- Caching

- Celery
- Email integration
- File storage
- Environment configuration
- Async views
- Reusable apps

Professional Backend Skills

- Git and GitHub
- PostgreSQL
- Database indexes
- Performance profiling
- Deployment
- HTTPS and production security
- Static/media serving
- Docker
- CI/CD
- Monitoring
- Backups
- README and API documentation
- Practical projects

Recommended Learning Order

1. Python fundamentals
2. Django setup and project structure
3. URLs, views and templates
4. Models, migrations and ORM
5. CRUD and forms
6. Admin and authentication
7. Relationships and query optimization
8. Django REST Framework
9. Serializers, API views and permissions
10. API testing and security
11. PostgreSQL and deployment

12. Caching and background tasks
13. Docker and CI/CD
14. Build, test and deploy real-world projects

Final Note

এই notes-এ Django-এর core concepts থেকে advanced backend development পর্যন্ত consolidated roadmap, explanations এবং code examples রাখা হয়েছে। প্রতিটি topic-এর mastery-এর জন্য নিজে code লেখা, tests করা, errors debug করা এবং project deploy করা জরুরি।

Official Documentation

- Django: https://docs.djangoproject.com/
- Django REST Framework: https://www.django-rest-framework.org/
- Python: https://docs.python.org/3/
- PostgreSQL: https://www.postgresql.org/docs/
- Docker: https://docs.docker.com/

Practice Goal: একটি Django CRUD application তৈরি করো, সেটিকে DRF API-তে রূপান্তর করো, authentication ও permissions যোগ করো, automated tests লেখো এবং production-এ deploy করো।